Eliminate dead code:

Aim:

To write and execute a C program to perform **Dead Code Elimination** on Three Address Code (TAC) and display the optimized code.

Algorithm:

1. Start the program.
2. Read the Three Address Code (TAC) lines until `"END"` is encountered, storing each line and splitting it into Left-Hand Side (LHS) and Right-Hand Side (RHS).
3. Initialize all statements as dead (`live = 0`).
4. Traverse the statements in reverse order (from bottom to top).
5. Mark a statement as `live` if it has no LHS variable or if its LHS matches critical output variables (such as `x` or `y`).
6. For every statement marked as `live`, scan its RHS for variables and mark their preceding definitions in the code as `live`.
7. Print only the statements that are marked as `live`.
8. Stop the program.

Pseudocode :

- Read lines into code[], extracting lhs[] and rhs[] until "END" is read.
- Loop i from $n-1$ down to 0:
- If lhs[i] is empty OR lhs[i] == "x" OR lhs[i] == "y", set live[i] = 1.
- If live[i] == 1:
- Extract all identifiers from rhs[i].
- For each identifier, find its latest definition j (where $j < i$ and lhs[j] == identifier) and set live[j] = 1.
- Loop i from 0 to $n-1$:
- If live[i] == 1, print code[i].

Code:

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>

#define MAX 100
#define LEN 100

int main()
{
   char code[MAX][LEN];
   char lhs[MAX][LEN];
   char rhs[MAX][LEN];
   int live[MAX] = {0};
   int n = 0, i, j;
   char line[LEN];

   printf("Enter Three Address Code statements.\n");
   printf("Enter END to finish:\n");

   while (n < MAX && fgets(line, sizeof(line), stdin))
   {
      line[strcspn(line, "\n")] = '\0';

      if (strcmp(line, "END") == 0)
         break;

      if (strlen(line) == 0)
         continue;

      strcpy(code[n], line);

      char *equal = strchr(line, '=');

      if (equal != NULL)
      {
         *equal = '\0';

         sscanf(line, "%99s", lhs[n]);
         strcpy(rhs[n], equal + 1);
      }
      else
      {
```

```
            lhs[n][0] = '\0';
            rhs[n][0] = '\0';
        }

        n++;
    }

    /* Perform Dead Code Analysis backwards */
    for (i = n - 1; i >= 0; i--)
    {
        /* Preserve statements with no LHS assignment or critical outputs (x, y) */
        if (lhs[i][0] == '\0' || strcmp(lhs[i], "x") == 0 || strcmp(lhs[i], "y") == 0)
        {
            live[i] = 1;
        }

        /* Mark earlier statements that define variables used in this live RHS */
        if (live[i])
        {
            char *p = rhs[i];

            while (*p)
            {
                if (isalpha((unsigned char)*p) || *p == '_')
                {
                    char var[LEN];
                    int k = 0;

                    while (isalnum((unsigned char)*p) || *p == '_')
                    {
                        var[k++] = *p++;
                    }

                    var[k] = '\0';

                    for (j = i - 1; j >= 0; j--)
                    {
                        if (strcmp(lhs[j], var) == 0)
                        {
                            live[j] = 1;
                            break;
                        }
                    }
                }
```

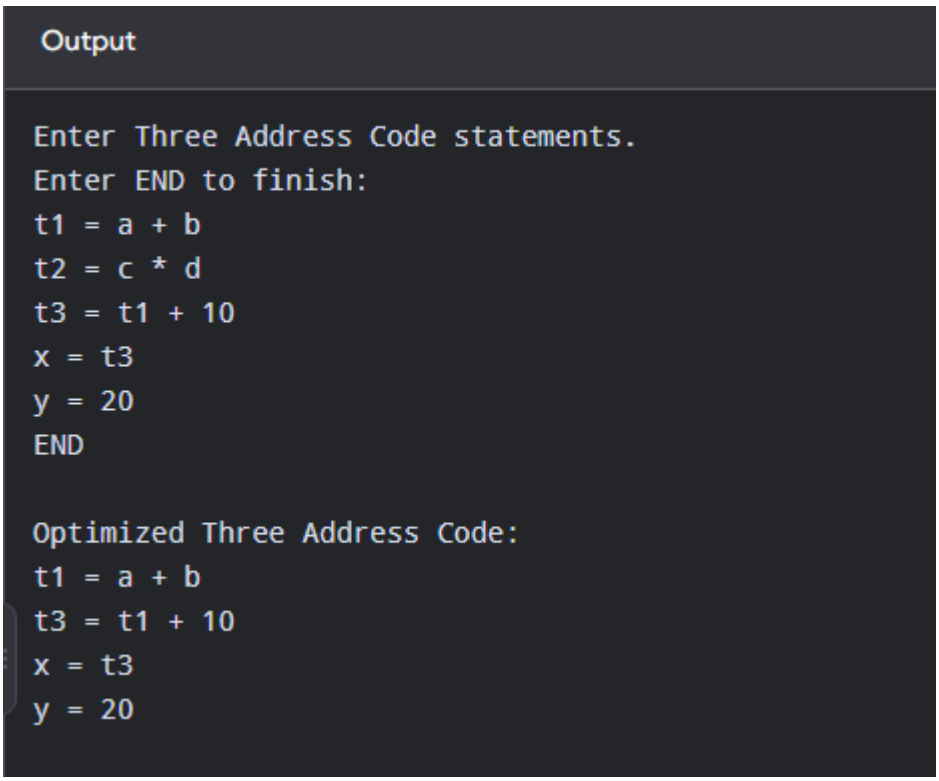
```
            else
            {
               p++;
            }
         }
      }
   }

   printf("\nOptimized Three Address Code:\n");

   for (i = 0; i < n; i++)
   {
      if (live[i])
         printf("%s\n", code[i]);
   }

   return 0;
}
```

OUTPUT:



Result:

Thus, the C program to implement Dead Code Elimination was successfully written, executed, and the output was verified.